Version Control Guidelines

Jose Velazquez Saenz

Bellevue University

CSD380: DevOps

Professor Sue Sampson

June 19, 2026

Version Control Guidelines

At its core, version control is a modern software development practice that allows developers to track changes, collaborate with others, review code, and revert to earlier versions when problems occur. A version control system records the history of changes to files over time, helping teams understand what changed, who made the change, and why. Today, Git is one of the most widely used version control systems because it supports distributed development, branching, merging, and cross-team collaboration. Version control is not only useful for large companies; it is also valuable for students, small teams, and individual developers. It creates a safer and more organized development process.

One common guideline across the sources is to make small, meaningful changes. GitLab's version control best practices recommend making incremental changes, keeping commits atomic, using branches, and writing descriptive commit messages. Atomic commits are important because each commit should represent a single clear change rather than several unrelated changes. This makes it easier to review code, find bugs, and roll back changes when needed. Google explains that smaller changes are easier to review, easier to design well, and less likely to block other work. These two sources agree that smaller changes improve code quality and make collaboration easier.

Another guideline is to use branches properly. GitHub explains that branches allow developers to work on features, bug fixes, or experiments in a contained area of the repository. This keeps unfinished work separate from the main branch. Atlassian's feature branch workflow also recommends creating temporary branches for development or testing. These branches can be pushed to the central repository so other developers can view or collaborate on the work without affecting the official code. GitHub and Atlassian both agree that branches improve safety and teamwork. However, in the Atlassian source, they also discuss Gitflow, which uses multiple long-running branches. While this can work well for some teams, newer DevOps practices often prefer simpler workflows and smaller branches.

A third major guideline is to use pull requests and code reviews. GitHub's pull request documentation explains that pull requests allow developers to propose changes, discuss them, review them, and address issues before merging into the main codebase. The main purpose of code review is to improve the overall health of the codebase over time. These sources show that version control is not only about saving code. It is also about communication and quality control. Pull requests give team members a place to explain changes, ask questions, catch mistakes, and make sure the code meets project standards before it becomes part of the official project.

Commit messages are another standard guideline and should be clear and descriptive. A good commit message helps future developers understand the purpose of a change. This is important because software projects often last months or years, and the people working on them may change over time. A vague message does not explain what was fixed or why. A better message would describe the actual change. Clear commit messages make the project history more useful.

When comparing the guidelines, the biggest similarity is that all three sources focus on reducing risk. GitLab emphasizes small commits, branches, and descriptive messages. GitHub emphasizes branches and pull requests. Google emphasizes small changes and code review. Each guideline is meant to make software changes easier to understand, test, review, and correct. The main difference is that some sources focus more on the tool itself, while others focus more on the team process. GitHub and GitLab provide practical Git-based guidance, while Google focuses more on engineering habits and code review culture.

Some older version control guidelines are less relevant today, depending on the project type. For example, older centralized version control systems often relied on stricter locking or checkout models where only one person could work on a file at a time. This is less common in modern software development because Git allows developers to work independently and merge changes later. Another guideline that may be less relevant today is the use of long-lived branches for every stage of

development. Gitflow can still work for large release-based teams, but many modern teams prefer shorter-lived branches. Atlassian's trunk-based development guidance recommends small batches and automated testing. This shows that modern version control practices are moving toward faster integration and smaller changes.

For me, the most important version control guidelines include committing small, focused changes; using clear messages; branching for new features, bug fixes, and experiments; and, most importantly, keeping the main branch stable. I selected these guidelines because they are the foundational ideas of version control. Breaking commits into smaller, easier-to-review pieces and including clear messages helps the project move forward. Branches make version control so useful for large or small teams because there is never a worry that what you code could affect the main branch negatively; they allow more ideas to be tested. Pull requests or code reviews before merging help catch errors early and allow team members to share knowledge. Finally, a stable branch makes it easier to release, test, and deploy the project.

I selected these guidelines because they support the main goals of version control: safety, collaboration, quality, and accountability. Version control should not simply be treated as a place to store code. It should be part of a complete development process that helps teams work together and reduce mistakes. Good version control habits make it easier to understand project history, review changes, recover from errors, and deliver software more confidently. Overall, the most effective version control guidelines encourage small changes, clear communication, frequent review, and a stable codebase.

References

Atlassian. (2026). *Git Feature Branch Workflow* . Atlassian git tutorial. https://www.atlassian.com/git/tutorials/comparing-workflows/feature-branch-workflow

Bartlett, R. A. (2023, August 24). *Google guidance on code review*. Better Scientific Software. https://bssw.io/items/google-guidance-on-code-review

github docs. (2026). *About branches* . https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-branches

OpenAI. (2026). ChatGPT (40 mimi). https://chat.openai.com/chat

Small-cls. Google. (2024). https://github.com/google/eng-practices/blob/master/review/developer/small-cls.md

What are git version control best practices?. GitLab. (2026). https://about.gitlab.com/topics/version-control/version-control-best-practices/

Zettler, K. (2026). *Trunk-based development*. Atlassian. https://www.atlassian.com/continuous-delivery/continuous-integration/trunk-based-development